CH 2 — Classes

Luke Lu • 2026-10-06

Intro to Class and Objects

> 📘 **Definition — Classes and Objects**
>
> Imperative Programming
>
> - Procedural Programming: data + standalone functions
> - Object-Oriented Programming: put the data and the functions that operate that data within unit or construct
>   - The construct is called **class**
>   - An instance/value of a class is called an **object**

Example:

A simple class called Student(Program "06-classes/version1.cc/student.cc"):

```
struct Student{
  int assignment, mt, final;
  float grade();
}

float Student::grade(){ //In the scope of student Class, the function grade
   return 0.4 * assignment + 0.2 * mt + 0.4 * final;
}
```

Note: in C++, `struct` can have functions within, and C++ `structs` are classes

Program "06-classes/version2.cc" is the more formal and preferred way of usage.

```
Student billy{90, 55, 45}; //C-style  initialization
//billy is a student object

float res1 = billy.grade(); //grade() has access to billy's marks

Student lily{95, 80, 85};

cout << lily.grade(); //grade() has access to lily's mark
```

> 📘 **Definition — Methods**
>
> A "function" part of a class is called "**methods**", which is commonly used
>
> A **method** can only be called by using an **object**
>
> Within the **method**, the method has access to the fields of that object.
>
> All **methods** have a hidden parameter named **`this`**, which is a pointer to the object use to call the method

Example:

```
billy.grade(); //"this" is equiv to "&billy", or interpret that their address is same
```

For the function, `grade` is actually

```
 float grade(){
  return this->assignment * 0.5 + this->mt * 0.2 + this->final 0.4;
};
```

We assume `this->` if not present and identifier undeclared

Object Initialization

In C++, we can write special methods to initialize objects

Previously,

1. C-style Initialization if there is no explicit constructor
   - Memory allocation
   - Field initialization in declaration order with parameter values in declaration order
     - In case of under-initialization, system does default initialization

       non-class/primitive data $\Longrightarrow$ assigned to `0`
2. C++ style Initialization if there is at least one explicit constructor
   - Memory allocation
   - Field initialization in declaration order
     - Primitives are not initialized
     - Classes are being called with **default constructors**
   - **Constructor** calls
     - Cannot default initialize in C-style
     - Logging
     - Constructor/function overloading
     - Sanity checks, restriction on input

> 📘 **Definition — Constructors Rules**
>
> - Same name as the class name
> - No return type
> - Default constructor does not have brackets upon calling
>
>   ```
>   CON a constructor(); //ILLEGAL
>   CON b constructor; //LEGAL
>   ```

```
struct Student{
  //...
  Student(int assign, int mt, int final){
    this->assign = assign;
    this->mt = mt;
    this->final = final;
  }
}
```

Member-Initialization List:

- field insitialization in declaration order of C++ style for fields named in the list
- use member-initilization list for all initializations
- use constructor for sanity checks, logging, etc.
- MIL processed in field declaration order. Not in MIL order.

```
struct id{
  string name;

  id;
  id(string name){ this->name = name;}
}
struct posn{
  int x, y;
  id uid;
  posn(int x, int y, string name): y{y*2}, x{x+1}, uid{name+ "?"} {}
}
```

For **ONE** parameter constructors:

```
struct Node {
  int data;
  Node *next;
  Node(int data):data{data}, next{nullptr}{}
}
Node m{4};
Node m = 4; //implicit conversion
foo(m);
foo(4); //One parameter constructor create implicit conversions
```

Note: implicit conversion for one parameter constructors could be useful and also be troublesome. We can disable implicit conversion by keyword: `explicit`

```
struct Node {
  int data;
  Node *next;
  explicit Node(int data):data{data}, next{nullptr}{}
}
Node m{4};
Node m = 4; //ILLEGAL
foo(m);
foo(4); //ILLEGAL
```

Class Methods

A class comes with a default constructor and the **Big 5**:

- Copy Constructor
- Copy Assignment Operator
- Destructor
- Move Constructor
- Move Assignment Operator

Definition — Copy Constructor

Copy Constructor creates a copy that is a new instance. It is used

- when constructing an object as a copy of another object
- called when an object is passed by value
- called when an object is return by value

Example:

1. Default implementation

```
//default hidden implementation
Student::Student(const Student &other) mt{other.mt}, final{other.final} {}
//must pass by reference
//Sometime, the default implemenation is not suitable

Student s1{90, 98, 33};
Student s2{s1}; //s2 = s1; the copy constructor
```

2. Custom implementation

```
struct Node {
  int data;
  Node *next;
  Node (int data, Node *next): data{data}, next{next}{}
  Node(const Node &other): data{other.data}, next{other.next}{}

  Node::Node(const Node &other) : \
  data{other.data}, next{other.next?new Node {*other.next}:nullptr;} {}

};

Node *p new Node{1, new Node {2, new Node {3, nullptr}}}
//p->1->2->3->null

Node m{*p}; //copy constructor the default

Node *q = new Node{*p}; //copy constructor
// Before having custom implementation
//m->1->2(of p)->3(of p)->null(of p)
//q->1->2(of p)->3(of p)->null(of p)

// After having custom implementation
//m->1->2->3->null
//q->1->2->3->null
```

Definition — Copy Assignment Operator

Assignment operator

- updates and existing object rather than creating a new instance.
- is implemented as a method
- the default implementation does field for field assignment

Example:

1. Copy constructor vs Assignment operator

```
Student s1{50, 12, 32};
Student s2{s1}; //copy constructor
Student s3 = s1; //copy constructor
Student s4{50, 23, 45};
s3 = s2; //copy assignment operator
```

2. Implementation for `Node`

```
n1 = n2; //n1.operator= (n2);
n1 = n2 = n3; //assignment cascades, so operator= must return a Node reference

Node &Node::operator=(const Node &other){

  if(this == &other) return *this; //self-assignment check
  data = other.data;
  delete next;//avoid memory leak, since `next` could be pointing somewhere
  next = other.next?new Node{*other.next}:nullptr;

  return *this; //or return others;
}
```